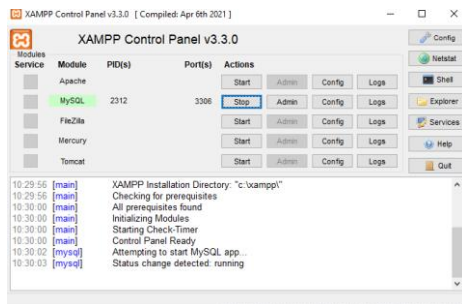


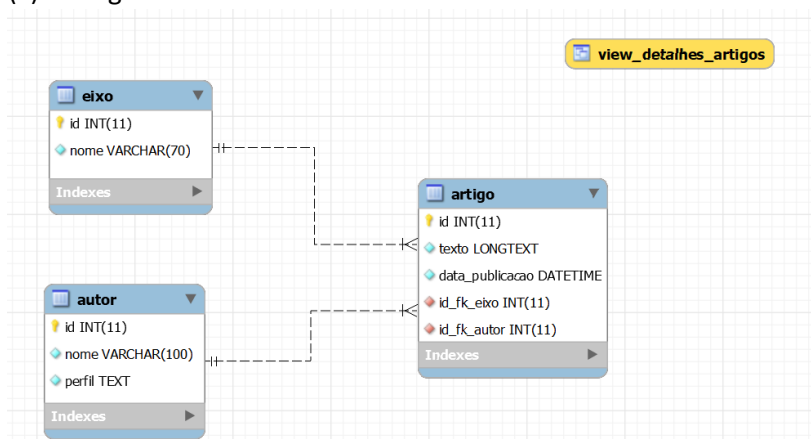
Atividade Laboratório Django

Julgue os itens e faça a atividade prática sem Inteligência Artificial

- () Django é um framework que usa o padrão MVT
- () Um CRUD tem apenas a ação de inserir no banco de dados
- () **XAMPP** é uma ferramenta usada para PHP , no SENAI será necessário inicia-lo



- () Ao abrir o MySQL para executar um SQL é necessário aparecer um trovão ativo
- () O diagrama esta com a cardinalidade correta



- () Ao criar uma classe artigo você poderá colocar pass, isso ajuda a tirar o erro de ausência de código.

```
2 class Autor(models.Model):
3     biografia = models.TextField()
4     email = models.EmailField()
5     def __str__(self):
6         return self.nome
7     class Meta:
8         db_table = 'autor'
9
10
11 class EixoTecnologia(models.Model):
12     nome = models.CharField(max_length=100)
13     def __str__(self):
14         return self.nome
15     class Meta:
16         db_table = 'eixo'
17
18 # crie a classe Artigo
19 class Artigo(models.Model):
20     pass
21
22
23
```

Considerando o código que você deverá implementar na classe Artigo no código da imagem abaixo:

```
# crie a classe Artigo
class Artigo(models.Model):
    texto = models.TextField()
    data_publicacao = models.DateTimeField(auto_now_add=True)
    id_fk_eixo = models.ForeignKey(
        EixoTecnologia,
        on_delete=models.CASCADE,
        db_column='id_fk_eixo'
    )
    id_fk_autor = models.ForeignKey(
        Autor,
        on_delete=models.CASCADE,
        db_column='id_fk_autor'
    )

    def __str__(self):
        return f"Artigo {self.id} - {self.data_publicacao}"

    class Meta:
        db_table = 'artigo'
```

Julgue os itens:

- () Artigo pode ser criado com o nome de artigo. A classe é case sensitive
- () A data de publicação não vai usar a data atual do sistema, o cliente vai ter que usar um componente Java Script de calendário para selecionar a data e a hora vai ser do sistema.
- () No trecho

```
id_fk_eixo = models.ForeignKey(
    EixoTecnologia,
    on_delete=models.CASCADE,
    db_column='id_fk_eixo'
)
```

Temos um exemplo de definição de chave estrangeira (ForeingKey) e ao deletar podemos afirmar as seguintes analogias:

As três opções principais:

CASCADE — "Se o pai morrer, os filhos morrem junto"

Deletou o Autor João?

→ Todos os artigos do João são deletados automaticamente.

Como apagar uma pasta no computador: some a pasta, somem os arquivos dentro.

PROTECT — "Não deixa deletar o pai enquanto tiver filho"

Tentou deletar o Autor João?

→ ERRO! "Não posso deletar, ele tem artigos cadastrados."

Como um banco que não deixa você fechar a conta se ainda tiver saldo.

SET_NULL — "O filho fica, mas fica órfão"

Deletou o Autor João?

→ Os artigos continuam existindo, mas o campo autor vira NULL (vazio).

Como um produto no e-commerce que fica no catálogo mesmo depois que o fornecedor sai do sistema.

⚠ Para usar, o campo precisa aceitar nulo: `null=True, blank=True`

Resumo visual:

Situação	Opção ideal
Artigo sem autor não faz sentido	<code>CASCADE</code> ou <code>PROTECT</code>
Quero manter o artigo mesmo sem autor	<code>SET_NULL</code>
Quero impedir exclusão acidental	<code>PROTECT</code>

() Para o nosso exemplo o cascade é a melhor opção

() No código abaixo foi utilizado o recurso de f-strings e a tabela não terá o nome padrão gerado pelo django.

```
def __str__(self):  
    return f"Artigo {self.id} - {self.data_publicacao}"  
  
class Meta:  
    db_table = 'artigo'
```

() Com esse comando eu faço o makemigrations no admin

```
python manage.py makemigrations motorartigos
```

() O comando abaixo mostra erros na geração dos migrations

```
Migrations for 'motorartigos':
  motorartigos\migrations\0001_initial.py
    - Create model Autor
    - Create model EixoTecnologia
    - Create model Artigo
```

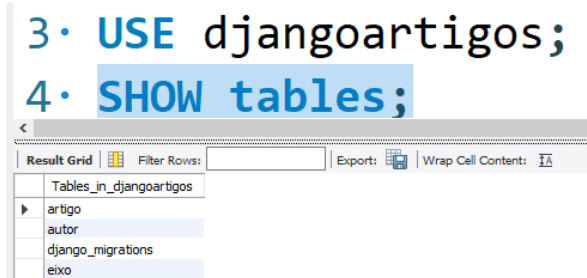
() Esse comando migrate vai aplicar os migrations do aplicativo no banco de dados criado do MySQL

```
python manage.py migrate motorartigos
```

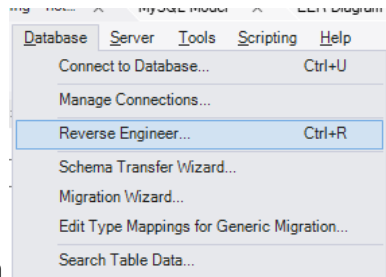
() Na imagem abaixo você pode visualizar que não teve sucesso a aplicação dos migrations pelo comando anterior do migrate

```
Operations to perform:
  Apply all migrations: motorartigos
Running migrations:
  Applying motorartigos.0001_initial... OK
```

() Ao abrir o MySQL o comando show tables irá executar e mostrar as tabelas criadas

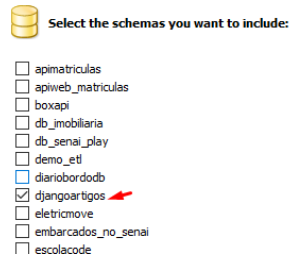


() No comando anterior foi criado uma tabela a mais de forma desnecessária “django_migrations” ela deve ser apagada urgentemente pois se trata de um vírus no computador.



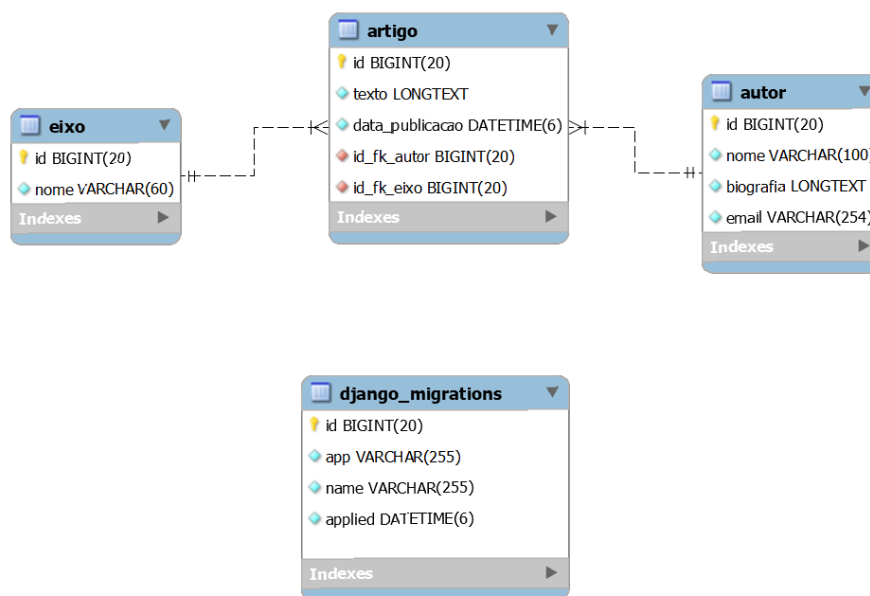
() Ao clicar em

e posteriormente marcar o banco de dados



utilizado

é gerado um diagrama MER:



() Para criar um editor de texto no painel de admin do Django vou ter que instalar uma biblioteca popular

pip install django-tinymce

() É uma boa prática recomendada após o comando executar um **pip freeze > requirements.txt**

() Esse print mostra que o django-tinymce foi instalado com sucesso

```

Download django_tinymce-5.0.0-py3-none-any.whl (1.4 MB)
 1.4/1.4 MB 6.6 MB/s 0:00:00
Download django-5.2.15-py3-none-any.whl (8.3 MB)
 8.3/8.3 MB 9.5 MB/s 0:00:00
Installing collected packages: django, django-tinymce
  Attempting uninstall: django
    Found existing installation: Django 4.1
    Uninstalling Django-4.1:
      Successfully uninstalled Django-4.1
    Successfully installed django-5.2.15 django-tinymce-5.0.0
  
```

() É correto afirmar que o código abaixo marcado em vermelho foi inserido no **models.py**

```
24 INSTALLED_APPS = [  
25     'django.contrib.admin',  
26     'django.contrib.auth',  
27     'django.contrib.contenttypes',  
28     'django.contrib.sessions',  
29     'django.contrib.messages',  
30     'django.contrib.staticfiles',  
31     'motorartigos',  
32 ]  
33  
34  
35 TINYMCE_DEFAULT_CONFIG = {  
36     'height': 500,  
37     'menubar': True,  
38     'plugins': [  
39         'advlist autolink lists link image charmap preview anchor',  
40         'searchreplace visualblocks code fullscreen',  
41         'insertdatetime media table paste code wordcount'  
42     ],  
43     'toolbar': (  
44         'undo redo | formatselect | bold italic underline strikethrough | '  
45         'alignleft aligncenter alignright alignjustify | '  
46         'bullist numlist outdent indent | link image | '  
47         'forecolor backcolor | code fullscreen'  
48     ),  
49 }
```

() Podemos afirmar que o comando abaixo marcado de vermelho vai instalar o aplicativo tinymce.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'motorartigos',  
    'tinymce',  
]
```

() Utilizar o comando

```
(.venv) D:\Users\ALUNO\Desktop\cursodjango>pip freeze > requirements.txt
```

Vai funcionar corretamente

```
(.venv) D:\Users\ALUNO\Desktop\cursodjango>pip list
Package            Version
-----
asgiref            3.11.1
Django             5.2.15
django-tinymce     5.0.0
mysqlclient        2.2.8
pip                26.1.2
python-dotenv      1.2.2
setuptools         57.4.0
sqlparse           0.5.5
typing_extensions  4.15.0
tzdata             2026.2
```

() o comando pip list é indicado para mostrar apenas recursos do django.

() Esse comando serve para importar a classe HTMLField do módulo **tinymce**.

```
1 from django.db import models
2 from tinymce.models import HTMLField
3 class Autor(models.Model):
4     nome = models.CharField(max_length=100)
5     biografia = models.TextField()
6     email = models.EmailField()
7     def __str__(self):
8         return self.nome
9     class Meta:
10         db_table = 'autor'
11
12 class EixoTecnologia(models.Model):
13     nome = models.CharField(max_length=60)
14     def __str__(self):
```

() É correto afirmar que a mensagem abaixo explica sobre compatibilidades de bibliotecas versus frameworks

```
Successfully uninstalled Django-5.2.15
ERROR: pip's dependency resolver does not currently take i
django-tinymce 5.0.0 requires django>=4.2, but you have dj
Successfully installed django-4.1
```

() Ao instalar a versão 4.2 resolverá a incompatibilidade e o makemigrations funcionará corretamente.

```
(.venv) D:\Users\ALUNO\Desktop\cursodjango>python manage.py makemigrations motorartigos
Migrations for 'motorartigos':
  motorartigos\migrations\0002_alter_artigo_texto.py
  - Alter field texto on artigo
```

() O tipo HTMLField

```
# crie a classe Artigo
class Artigo(models.Model):
    texto = HTMLField()
    data_publicacao = models.DateTimeField(auto_now_add=True)
    id_fk_eixo = models.ForeignKey(
        EixoTecnologia,
        on_delete=models.CASCADE,
        db_column='id_fk_eixo'
    )
    id_fk_autor = models.ForeignKey(
        Autor,
        on_delete=models.CASCADE,
        db_column='id_fk_autor'
    )
```

E dar um makemigrations e migrations respectivamente irá aparecer as mensagens de sucesso e no banco de dados terá a seguinte configuração:

7. describe artigo;

8

Result Grid						
Filter Rows:						
Export: Wrap Cell Content:						
	Field	Type	Null	Key	Default	Extra
▶	id	bigint(20)	NO	PRI	NULL	auto_increment
	texto	longtext	NO		NULL	
	data_publicacao	datetime(6)	NO		NULL	
	id_fk_autor	bigint(20)	NO	MUL	NULL	
	id_fk_eixo	bigint(20)	NO	MUL	NULL	

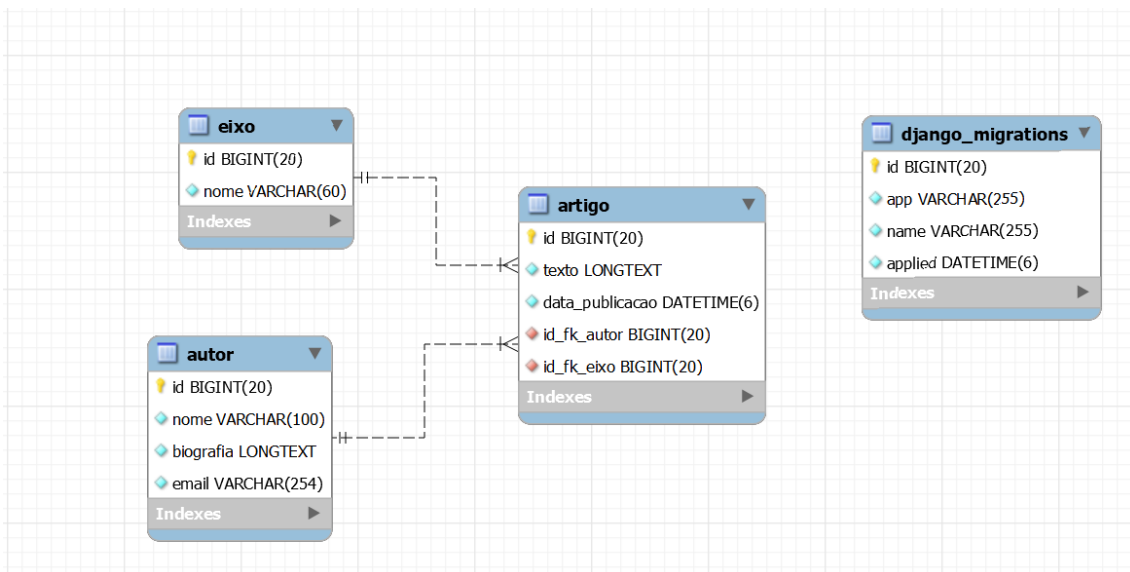
() Essa atualização será necessário para termos o autor, eixo e artigos funcionando corretamente no admin


```

1  from django.contrib import admin
2  from tinymce.widgets import TinyMCE
3  from django.db import models as db_models
4  from .models import Autor, EixoTecnologia, Artigo
5
6  @admin.register(Autor)
7  class AutorAdmin(admin.ModelAdmin):
8      list_display = ('id', 'nome', 'email')
9      search_fields = ('nome', 'email')
10
11  @admin.register(EixoTecnologia)
12  class EixoTecnologiaAdmin(admin.ModelAdmin):
13      list_display = ('id', 'nome')
14      search_fields = ('nome',)
15
16  @admin.register(Artigo)
17  class ArtigoAdmin(admin.ModelAdmin):
18      list_display = ('id', 'id_fk_autor', 'id_fk_eixo', 'data_publicacao')
19      search_fields = ('id_fk_autor__nome',)
20      list_filter = ('id_fk_eixo', 'data_publicacao')
21      formfield_overrides = {
22          db_models.TextField: {'widget': TinyMCE()}, # 🖱️ garante o editor
23      }

```

() O resultado final do MER é o seguinte



() Esse erro mostra que foi realizado o makemigrations e migrate no admin

```

db.query(q)
File "d:\Users\ALUNO\Desktop\cursodjango\.venv\lib\site-packages\MySQLdb\connections.py", line 253, in query
_mysql.connection.query(self, query)
django.db.utils.ProgrammingError: (1146, "Table 'djangoartigos.auth_user' doesn't exist")

(.venv) D:\Users\ALUNO\Desktop\cursodjango>

```

() Esse comando dá o makemigrations no admin

```
python manage.py makemigrations
```

() Esse comando mostra claramente o migrate sendo aplicado no admin

```
(.venv) D:\Users\ALUNO\Desktop\cursodjango>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, motorartigos, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying motorartigos.0002_alter_artigo_texto... OK
  Applying sessions.0001_initial... OK
```

() Esse é uma forma correta e segura de criar superusuário

```
python manage.py createsuperuser
Usuário (leave blank to use 'professor'): romulo
Endereço de email:
Password:
Password (again):
Esta senha é muito curta. Ela precisa conter pelo menos 8 caracteres.
Esta senha é muito comum.
Esta senha é inteiramente numérica.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

() Esse comando executa a aplicação

```
(.venv) D:\Users\ALUNO\Desktop\cursodjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

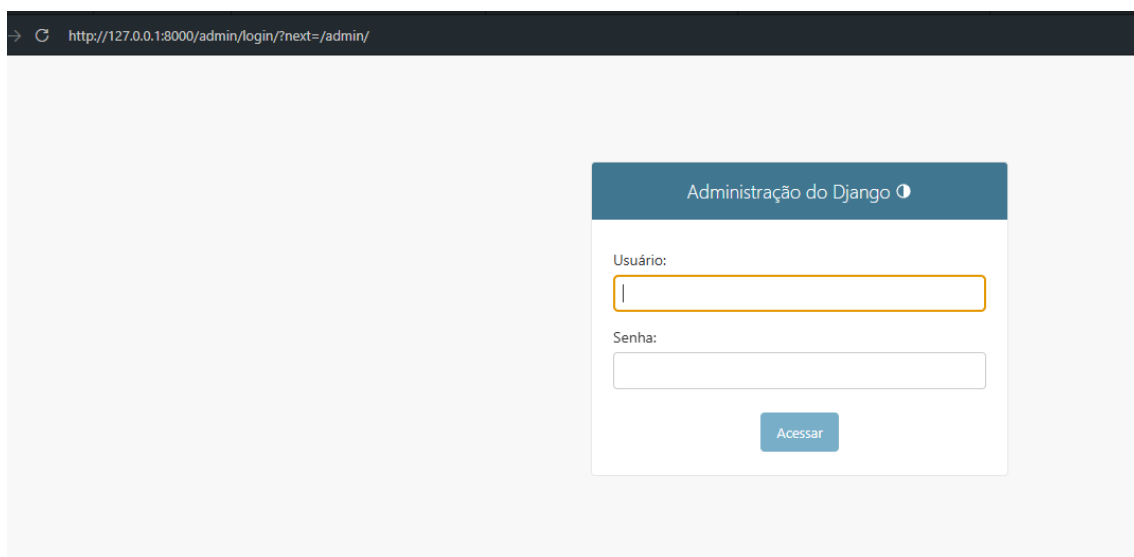
System check identified no issues (0 silenced).
June 24, 2026 - 12:32:29
Django version 4.2, using settings 'setup.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

() A primeira página é a seguinte:



The screenshot shows a web browser at the address `http://127.0.0.1:8000/`. The page has a dark red background. At the top left, there is a list of links: `fundamentos python`, `fundamentos redes`, `fundamentos nuvem`, `fundamentos IA`, `fundamentos Data Science`, and `fundamentos Django`. Below this list is the text **Django DF** and `Comunidade Django do Distrito Federal`. The main heading is **O melhor site de Django, IA e Data Science do Brasil**. Below the heading is a logo consisting of a black knot. Underneath the logo is the text **Autores Cadastrados** and `Nenhum autor encontrado.`. At the bottom right of the page, there is a footer that says `DANGODF © 2026`.

() Assim você acesso admin



The screenshot shows a web browser at the address `http://127.0.0.1:8000/admin/login/?next=/admin/`. The page is white with a blue header that says **Administração do Django**. Below the header, there is a login form. The form has two input fields: `Usuário:` and `Senha:`. Below the `Senha:` field is a blue button that says `Acessar`.

() Com usuário e senha entra na seguinte tela:

Administração do Django

Administração do Site

AUTENTICAÇÃO E AUTORIZAÇÃO

Grupos	+ Adicionar	✎ Modificar
Usuários	+ Adicionar	✎ Modificar

MOTORARTIGOS

Artigos	+ Adicionar	✎ Modificar
Autores	+ Adicionar	✎ Modificar
Eixo tecnologias	+ Adicionar	✎ Modificar

Ações recentes

Minhas Ações

Nenhum disponível

() Assim você adiciona um autor

Adicionar autor

Nome:

romulo

Biografia:

o melhor de todos

Email:

romulodf@gmail.com

SALVAR

Salvar e adicionar outro(a)

Salvar e continuar editando

() Assim você adiciona um eixo tecnológico

Adicionar eixo tecnologia

Nome:

iniciando em dados

SALVAR

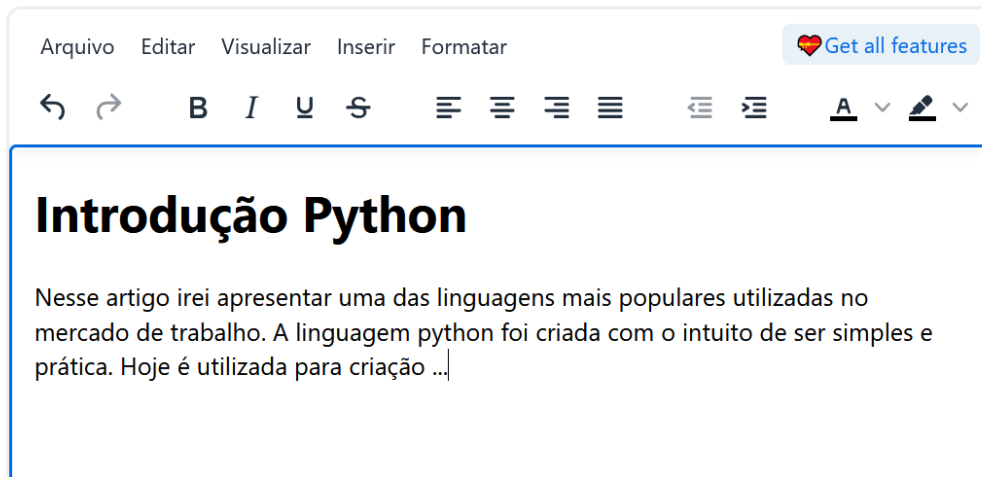
Salvar e adicionar outro(a)

Salvar e continuar editando

() A tela abaixo mostra que o módulo e a classe externa ao django foi instalada e utilizada com sucesso.

Adicionar artigo

Texto:



The screenshot shows a rich text editor with a menu bar (Arquivo, Editar, Visualizar, Inserir, Formatar) and a toolbar with icons for undo, redo, bold, italic, underline, strikethrough, bulleted list, numbered list, decrease indent, increase indent, text color, and background color. The main text area contains the title "Introdução Python" and a paragraph: "Nesse artigo irei apresentar uma das linguagens mais populares utilizadas no mercado de trabalho. A linguagem python foi criada com o intuito de ser simples e prática. Hoje é utilizada para criação ...". A "Get all features" button is visible in the top right corner.

() O autor e eixo funcionou corretamente e o visual ficou bem intuitivo nível UX/UI Design



The screenshot shows a form with two fields. The first field is labeled "Id fk eixo:" and has a dropdown menu with a dashed line and a checkmark icon. The second field is labeled "Id fk autor:" and has a dropdown menu with a dashed line and the text "iniciando em dados". To the right of the first dropdown is a yellow pencil icon and a green plus icon.

Questão Final

Faça ajustes no visual e no código do admin para uma melhor utilização de UX/UI Design.